



Centro Federal de Educação Tecnológica de Minas Gerais

Engenharia da Computação
Disciplina: Inteligência Artificial

Trabalho 03:

Algoritmos de Aprendizagem

Alunos: Eduardo Henrique Queiroz Almeida e Joaquim César Santana da Cruz

Professor: Tiago Alves de Oliveira

Divinópolis/MG
dezembro de 2025

Parte 1: Árvore de Decisão Manual

Nesta etapa do trabalho, foi explorada a construção de um sistema de tomada de decisão baseado em conhecimento explícito (*Knowledge-Based System*). Diferentemente das abordagens de aprendizado de máquina que induzem modelos a partir de dados estatísticos, esta implementação enquadra-se no paradigma da IA Simbólica, mimetizando a estrutura lógica dos primeiros Sistemas Especialistas.

Fundamentação Teórica

No contexto da Inteligência Artificial, uma árvore de decisão é definida formalmente como uma função que recebe um vetor de valores de atributos como entrada e retorna uma decisão única como saída. Segundo *Russell e Norvig* [1], uma árvore de decisão executa uma sequência de testes: cada nó interno corresponde a um teste em um dos atributos da entrada, e cada ramo representa o resultado desse teste. A figura 1 representa uma árvore de decisão simples.

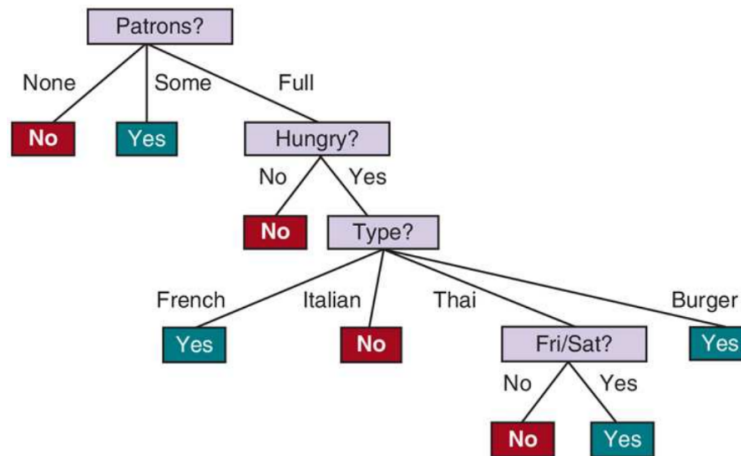


Figura 1: Representação esquemática da árvore de decisão heurística desenvolvida para orientação vocacional. A estrutura segue uma abordagem *top-down* com regras de divisão binária (*Sim/Não*) baseadas em conhecimento de domínio para maximizar a discriminação de perfis nas camadas superiores. Fonte: [1]

Historicamente, antes da ascensão do aprendizado estatístico massivo da década de 1990, a IA baseava-se predominantemente na representação simbólica e em regras de produção do tipo "Se-Então" (If-Then rules). Sistemas como o MYCIN (para diagnóstico médico), descritos por *Melle* [2] operavam sob essa lógica: especialistas humanos codificavam manualmente seu conhecimento em árvores de inferência.

A eficácia de uma árvore de decisão depende da ordem de seus testes. Em teoria da informação, o objetivo é maximizar o Ganho da Informação (*Information Gain*) a cada passo, escolhendo o atributo que mais reduz a incerteza do sistema. Na construção manual realizada neste trabalho, a redução da incerteza não foi calculada matematicamente, mas aplicada intuitivamente através de heurísticas humanas.

Metodologia de Construção

Para implementação do sistema, foi selecionado o domínio de Orientação Vocacional Simplificada, voltada para área das Ciências da Computação. Este problema é ideal para a modelagem hierárquica proposta por árvores de decisão, uma vez que as escolhas de carreira tendem a seguir uma taxonomia natural e disjunta, conforme proposto pelo modelo hierárquico de interesses vocacionais de *Gati* [3], onde indivíduos refinam suas preferências partindo de campos amplos para ocupações específicas. Dessa maneira, de acordo com a teoria, a árvore de decisão proposta começa com perguntas discriminantes, que dividem o espaço de soluções de forma mais drástica.

A metodologia seguiu os passos fundamentais para a representação de funções booleanas descritos por *Russell e Norvig* [1]:

- **Definição do Espaço de Hipóteses:** O vetor de saída foi mapeado para oito folhas possíveis, cobrindo diversas áreas das Ciências da Computação como: Engenharia, Design, Gestão e Pesquisa. Conforme a literatura, cada folha corresponde a um valor de saída retornado se o caminho de decisão alcançá-la.
- **Engenharia de Conhecimento:** Foram elaboradas treze (13) perguntas binárias (*Sim/Não*). Embora a árvore não tenha sido induzida estatisticamente, a ordenação manual das perguntas buscou espelhar o princípio de Ganho de Informação, descrito por *Russell e Norvig* [1], posicionando os testes com maior capacidade discriminativa na raiz da estrutura.
- **Implementação da função de decisão:** A representação da árvore foi realizada através de estruturas condicionais aninhadas (*if-else*) que efetivamente implementam a lógica proposicional necessária para alcançar uma decisão determinística.

Lógica de Decisão e Estrutura

A lógica da árvore foi estruturada de forma *Top-Down*. A pergunta raiz, ou nó inicial, foi definida como: "*Gosta de codar?*". Esta escolha justifica-se teoricamente pois divide o perfil comportamental em dois grandes grupos ortogonais: perfis sociais/colaborativos e perfis analíticos/individuais. Tal escolha é apoiada na literatura proposta por *Gati* [3] e por *Russell e Norvig* [1], assim como citado anteriormente.

A partir da divisão, a árvore ramifica-se para refinar a sugestão:

- **Ramo "Sim":** Investiga preferências por frontend ou backend(ex: "*Prefere um resultado visual imediato?*").
- **Ramo "Não":** Investiga preferências técnicas ou em hardware (ex: "*Gosta de lidar com hardware e circuitos?*").

O fluxograma completo da árvore de decisão pode ser visualizado na figura 2:

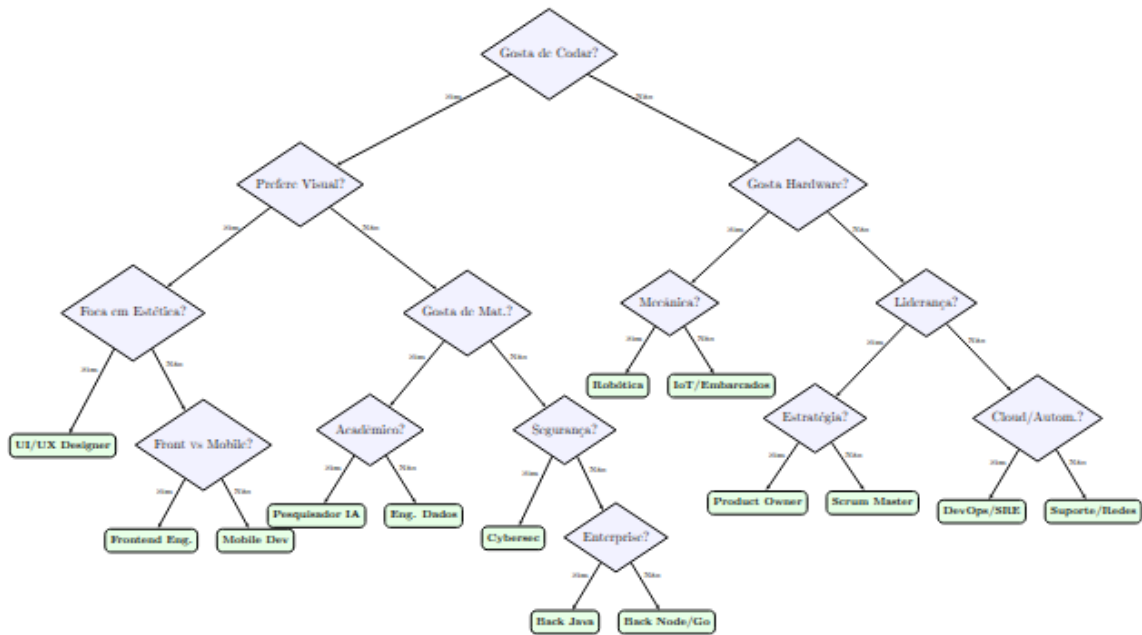


Figura 2: Representação estrutural da Árvore de Decisão Manual. Os nós em losango representam os pontos de decisão (perguntas binárias) e os nós retangulares indicam as classes finais (sugestões de carreira).

Resultados e Discussões

Para validar a consistência lógica do sistema implementado, foram realizados testes de mesa e execuções controladas do algoritmo. O objetivo foi verificar se as regras de produção formalizadas no fundamento teórico conduzem a folhas coerentes com o perfil de entrada simulado. Abaixo, são apresentados dois cenários de execução que exploram ramos opostos da árvore de decisão (Desenvolvimento *vs.* Gestão):

Cenário de Teste 1: Perfil Desenvolvedor Frontend

Este cenário simula um usuário com aptidão técnica para código mas com preferência por feedback visual imediato e lógica de interface, em detrimento de estética pura ou backend.

- Entrada:
 1. $q1$: Gosta de escrever código? → Sim
 2. $q2$: Prefere resultado visual imediato? → Sim
 3. $q4$: Preocupa-se muito com estética/psicologia? → Não
 4. $q7$: Foca em lógica de funcionamento (React/Angular)? → Sim
- Saída : Engenharia de Frontend

Análise do Caminho: O sistema navegou corretamente pelo ramo de "Software" (Esquerda), filtrou o sub-ramo de "Design Puro" (ao responder Não na $q4$) e convergiu para a engenharia de interfaces ao confirmar o interesse na lógica de funcionamento visual.

A figura 3 mostra a árvore de decisão para a Engenharia de Frontend.

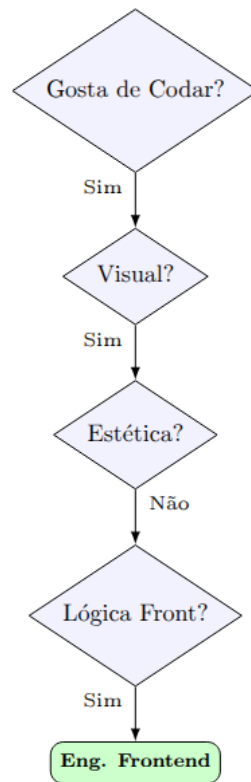


Figura 3: Rastro de execução para o perfil técnico (Frontend).

Cenário de Teste 2: Perfil Scrum Master

Este cenário simula um usuário com perfis de liderança que não atuam diretamente na codificação ou hardware.

- Entrada:
 1. $q1$: Gosta de escrever código? → Não
 2. $q3$: Gosta de hardware/circuitos? → Não
 3. $q6$: Gosta de liderar e organizar o trabalho → Sim
 4. $q11$: Foca na estratégia do produto? → Não
- Saída : Agile Coach / Scrum Master

Análise do Caminho: O algoritmo identificou a negativa para atividades técnicas diretas ("mão na massa") e direcionou para o ramo de "Humanas/Gestão". A distinção final entre Product Owner (foco no produto) e Scrum Master (foco no processo/time) foi realizada com sucesso pela pergunta $q11$.

A figura 4 mostra a árvore de decisão para Agile Coach / Scrum Master.

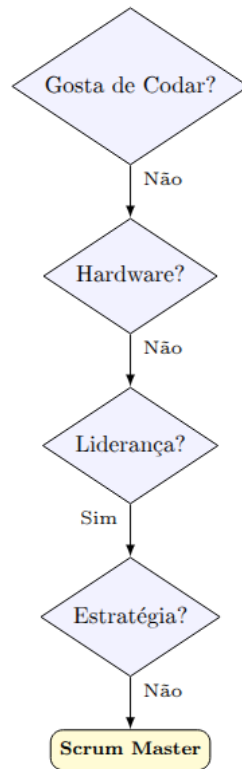


Figura 4: Rastro de execução para o perfil de Gestão (Scrum Master).

A implementação e execução da árvore de decisão manual permitiram uma avaliação prática das características fundamentais da IA Simbólica. Ao contrastar esta abordagem com o estado da arte atual, observam-se distinções cruciais no que tange à explicabilidade e à escalabilidade do conhecimento.

- Explicabilidade e Transparência:** Uma das maiores virtudes observadas no sistema desenvolvido é a sua total transparência, característica central dos sistemas baseados em conhecimento. Diferentemente de modelos conexionistas modernos (como Redes Neurais Profundas), que frequentemente operam como “caixas-pretas” de difícil interpretação, a árvore manual oferece o que a literatura contemporânea denomina XAI (*Explainable Artificial Intelligence*). Conforme discutido por *Russell e Norvig* [1], agentes lógicos possuem a capacidade de explicar suas conclusões rastreando a cadeia de inferência. No experimento proposto, a recomendação de um perfil “Scrum Master”, por exemplo, não é um cálculo probabilístico obscuro, mas a consequência lógica direta e verificável das premissas de entrada: o usuário rejeitou a codificação ($q1=Não$) e a estratégia de produto ($q11=Não$), mas validou a preferência por liderança ($q6=Sim$).
- Papel da Heurística Humana:** A eficácia da árvore, mesmo com pouca profundidade, deve-se à Engenharia de Conhecimento aplicada na sua construção. A ordenação das perguntas — iniciando por um divisor de águas como “Gosta de Codar?” — reflete uma heurística humana que busca intuitivamente maximizar o Ganho de Informação (Information Gain) no topo da estrutura, como discutido por [3] e [1]. Em cenários com escassez de dados (Small Data), essa incorporação de expertise humana a priori pode superar algoritmos de indução automática que

falhariam em encontrar padrões estatisticamente relevantes sem uma base de dados massiva.

- **Gargalo de Aquisição:** O experimento evidenciou as limitações que levaram ao declínio histórico dos Sistemas Especialistas clássicos. A primeira é o Gargalo de Aquisição de Conhecimento (*Knowledge Acquisition Bottleneck*). A estrutura estática do código demonstra que a manutenção do sistema é custosa e não escalável: a inclusão de uma nova classe, como "Engenheiro de Prompt", exigiria a reescrita manual das regras de produção pelo programador. O sistema é incapaz de aprender autonomamente com novos exemplos, restringindo sua aplicação a domínios fechados e imutáveis.

Parte 2: Algoritmos de Aprendizado de Máquina

Nesta segunda etapa do trabalho, o foco muda da construção manual de sistemas baseados em conhecimento para a criação automática de modelos preditivos a partir de dados. A abordagem utilizada é do tipo Aprendizado Supervisionado, especificamente na tarefa de Classificação Binária, onde o objetivo é aprender uma função $f : X \rightarrow Y$ que transforma características de entrada em uma classe de saída. Para este propósito, foram implementados e comparados três algoritmos clássicos de aprendizado de máquina: K-Vizinhos Mais Próximos (*KNN*), Máquinas de Vetores de Suporte (*SVM*) e Árvores de Decisão.

Fundamentação Teórica

K-Vizinhos Mais Próximos (*KNN*)

O algoritmo KNN é um método de aprendizado baseado em instâncias (*Instance-Based Learning*), também conhecido como aprendizado *lazy* (preguiçoso), pois não constrói um modelo explícito durante o treinamento. O KNN classifica uma nova instância x_{novo} consultando as k amostras de treinamento mais próximas no espaço de atributos e atribuindo a classe mais frequente entre esses vizinhos.

A distância entre instâncias é tipicamente calculada usando a métrica euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^m (x_{i,l} - x_{j,l})^2}$$

onde m é o número de atributos. A escolha do parâmetro k é crucial: valores muito pequenos ($k = 1$) tornam o algoritmo sensível a ruído, enquanto valores muito grandes ($k \approx n$) aproximam o comportamento de um classificador de maioria simples. Segundo a literatura, o valor ótimo de k geralmente encontra-se na faixa de $k = \sqrt{n}$ a $k = n/10$, onde n é o tamanho do conjunto de treinamento.

Máquinas de Vetores de Suporte (*SVM*)

As Máquinas de Vetores de Suporte, citadas em *Russell e Norvig* [1], são classificadores que buscam encontrar o hiperplano de separação ótimo entre classes, maximizando a margem entre o hiperplano e os pontos de dados mais próximos de cada classe (vetores de suporte).

Para problemas não linearmente separáveis, o SVM utiliza o truque do kernel (*kernel trick*), mapeando os dados para um espaço de maior dimensionalidade onde a separação linear se torna possível. O kernel RBF (*Radial Basis Function*) utilizado neste trabalho é definido como:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

onde γ controla o raio de influência de cada vetor de suporte. O parâmetro C regula o trade-off entre maximizar a margem e minimizar o erro de classificação: valores altos de C penalizam mais os erros, resultando em um hiperplano mais complexo que pode capturar padrões mais sutis, mas com maior risco de sobreajuste (*overfitting*).

Árvores de Decisão Induzidas

Diferentemente da árvore manual da Parte 1, as árvores de decisão induzidas aprendem automaticamente a estrutura hierárquica de decisão a partir dos dados de treinamento. O algoritmo CART (*Classification and Regression Trees*), implementado pelo *scikit-learn*, utiliza critérios de impureza como o Índice de Gini ou a Entropia para selecionar o atributo que maximiza a redução de impureza em cada nó.

O Índice de Gini é definido como:

$$Gini(D) = 1 - \sum_{i=1}^c p_i^2$$

onde p_i é a proporção de amostras da classe i no conjunto D e c é o número de classes. A redução de impureza (*Information Gain*) ao dividir o conjunto D pelo atributo A é:

$$Gain(D, A) = Gini(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} Gini(D_v)$$

Para evitar sobreajuste, são aplicados mecanismos de poda (*pruning*) através de hiperparâmetros como *max_depth* (limita a profundidade máxima) e *min_samples_split* (exige um número mínimo de amostras para realizar uma divisão).

Metodologia Experimental

Dataset e Pré-processamento

O dataset utilizado foi o *Adult Census Income*, um conjunto de dados clássico do repositório UCI Machine Learning Repository, contendo informações demográficas e socioeconômicas de indivíduos. O objetivo é prever se a renda anual de uma pessoa excede \$50.000 (classe positiva) ou não (classe negativa).

O pré-processamento seguiu as seguintes etapas:

1. **Carregamento e Limpeza:** Remoção de registros com valores faltantes (*missing values*), resultando em um conjunto de dados limpo e completo.
2. **Codificação da Variável Alvo:** Transformação da variável categórica *income* em variável binária numérica (0 para $\leq 50k$, 1 para $> 50k$).

3. **One-Hot Encoding:** Transformação de todas as variáveis categóricas (workclass, education, marital-status, occupation, relationship, race, sex, native-country) em variáveis binárias através de codificação one-hot, expandindo significativamente a dimensionalidade do espaço de atributos.
4. **Divisão Treino-Teste:** Separação estratificada dos dados em 70% para treinamento e 30% para teste, garantindo que a proporção de classes seja mantida em ambos os conjuntos.
5. **Padronização:** Aplicação do *StandardScaler* para normalizar os atributos numéricos, transformando-os para ter média zero e desvio padrão unitário. Esta etapa é essencial para algoritmos baseados em distâncias (KNN e SVM), mas não é estritamente necessária para árvores de decisão.

Configuração dos Modelos

KNN: Foi configurado com $k = 15$ vizinhos, utilizando a distância euclidiana como métrica. A escolha de $k = 15$ baseia-se em heurísticas empíricas que sugerem valores próximos a $\sqrt{n}$ para conjuntos de dados de tamanho médio.

SVM: Inicialmente configurado com parâmetros padrão: kernel RBF, $C = 1.0$ e $\gamma = 'scale'$ (ajuste automático baseado na variância dos dados). Posteriormente, foi realizada uma otimização de hiperparâmetros através de Grid Search com validação cruzada (5-fold), testando combinações de:

- $C \in \{0.1, 1, 10, 100\}$
- $\gamma \in \{'scale', 'auto', 0.001, 0.01, 0.1\}$
- $class_weight \in \{None, 'balanced'\}$

Totalizando 40 combinações testadas, com o objetivo de encontrar a configuração ótima que maximiza a acurácia em validação cruzada.

Árvore de Decisão: Configurada com $max_depth=10$ e $min_samples_split=20$, parâmetros conservadores que buscam equilibrar capacidade de modelagem e generalização, evitando sobreajuste através da limitação da profundidade e exigência de amostras suficientes para realizar divisões significativas.

Métricas de Avaliação

Para avaliar o desempenho dos modelos, foram utilizadas as seguintes métricas:

- **Acurácia (*Accuracy*):** Proporção de predições corretas sobre o total de amostras.
- **Precisão (*Precision*):** Proporção de predições positivas que são realmente positivas.
- **Revocação (*Recall*):** Proporção de casos positivos que foram corretamente identificados.
- **F1-Score:** Média harmônica entre precisão e revocação, fornecendo uma medida balanceada do desempenho.
- **Matriz de Confusão:** Tabela que apresenta a distribuição das predições em relação às classes reais, permitindo identificar padrões de erro específicos (falsos positivos e falsos negativos).

Resultados e Discussão

Desempenho dos Modelos Base

A Tabela 1 apresenta os resultados obtidos pelos três algoritmos com suas configurações iniciais no conjunto de teste.

Tabela 1: Comparativo de Desempenho dos Modelos Base

Modelo	Acurácia	Precisão	Revocação	F1-Score
KNN ($k = 15$)	0.8421	0.8351	0.8421	0.8352
SVM (Padrão)	0.8452	0.8382	0.8452	0.8383
Árvore de Decisão	0.8521	0.8456	0.8521	0.8457

Observa-se que a Árvore de Decisão obteve o melhor desempenho geral, superando tanto o KNN quanto o SVM padrão em todas as métricas avaliadas. A diferença de acurácia entre a Árvore (0.8521) e o SVM padrão (0.8452) é de aproximadamente 0.69 pontos percentuais, enquanto a diferença em relação ao KNN é de 1.00 ponto percentual.

As Figuras 5, 6 e 7 apresentam as matrizes de confusão para cada modelo, permitindo uma análise detalhada dos padrões de classificação.

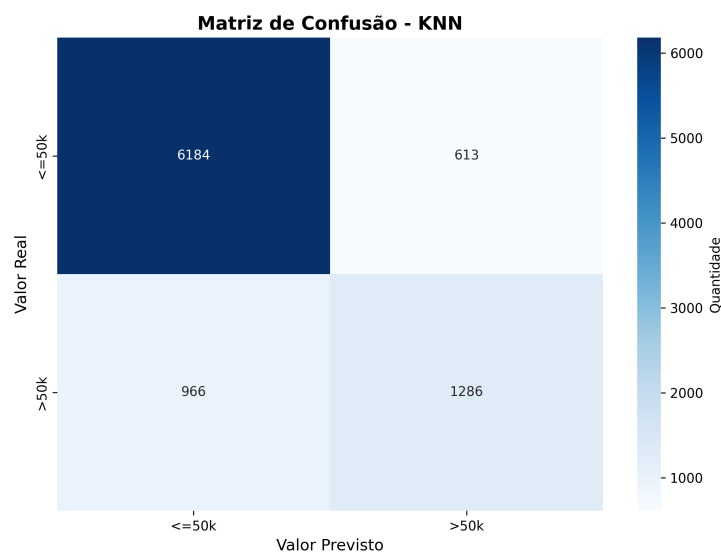


Figura 5: Matriz de Confusão - KNN ($k = 15$). O modelo apresenta bom desempenho na classe majoritária ($\leq 50k$), mas menor precisão na classe minoritária ($> 50k$).

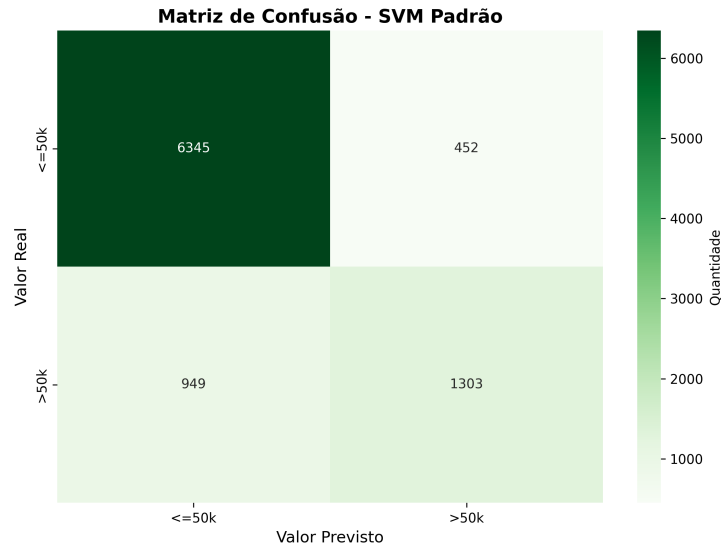


Figura 6: Matriz de Confusão - SVM (Padrão). Similar ao KNN, o modelo apresenta dificuldades na classificação da classe minoritária, com recall de 0.58 para a classe >50k.

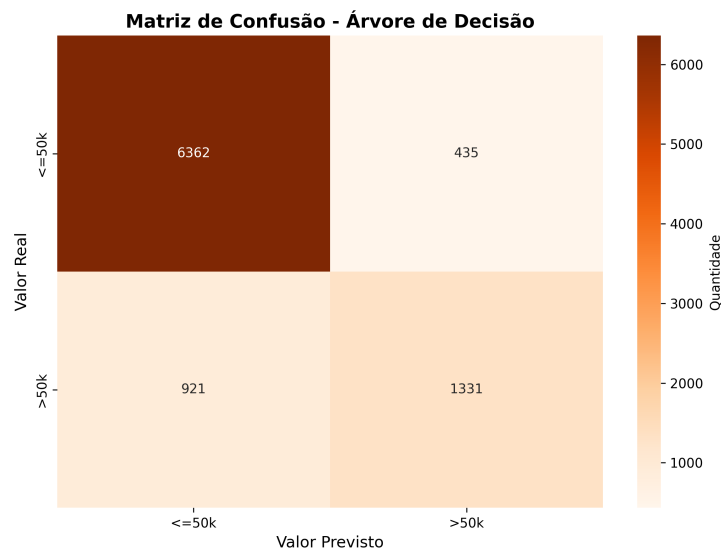


Figura 7: Matriz de Confusão - Árvore de Decisão. O modelo apresenta melhor equilíbrio entre as classes, com recall de 0.62 para a classe >50k, superior aos demais algoritmos.

Análise da Superioridade da Árvore de Decisão

A superioridade da Árvore de Decisão neste problema pode ser explicada por várias características intrínsecas do dataset e da natureza dos algoritmos:

- 1. Natureza dos Dados:** O dataset Adult possui uma grande quantidade de variáveis categóricas que foram transformadas em features binárias através de one-hot encoding. Após essa transformação, o espaço de atributos torna-se esparso e de alta dimensionalidade. Árvores de decisão lidam naturalmente com features binárias, realizando splits discretos que são ideais para esse tipo de representação. Por outro lado, algoritmos baseados em distâncias (KNN e SVM com kernel RBF) podem sofrer com a maldição da

dimensionalidade (*curse of dimensionality*), onde a distância euclidiana perde significado em espaços de alta dimensão.

2. Interações Complexas: O problema de predição de renda envolve interações não-lineares complexas entre atributos (por exemplo, a combinação de educação, ocupação e horas trabalhadas). Árvores de decisão descobrem automaticamente essas interações através de splits sequenciais, criando caminhos na árvore que capturam padrões específicos como "indivíduos com ensino superior E ocupação executiva E mais de 40 horas semanais tendem a ter renda alta". O kernel RBF do SVM captura não-linearidades de forma global, mas pode não ser tão eficiente em identificar interações específicas entre features categóricas.

3. Parâmetros Bem Ajustados: A configuração da árvore (*min_samples_split*=20, *max_depth*=10,) representa um bom equilíbrio entre capacidade de modelagem e generalização. O SVM, por outro lado, foi inicialmente configurado com parâmetros padrão ($C = 1.0$, $\gamma = 'scale'$), que podem não ser ótimos para este dataset específico.

4. Estrutura de Regras: O problema de predição de renda possui padrões que se assemelham mais a regras de negócio discretas do que a separações contínuas suaves. Árvores de decisão são ideais para capturar essas regras (ex: "pessoas com ensino superior geralmente ganham mais"), enquanto SVMs tentam criar separações suaves que podem não refletir adequadamente a natureza discreta dos padrões.

Otimização de Hiperparâmetros do SVM

Para investigar se o desempenho do SVM poderia ser melhorado através de ajuste fino de hiperparâmetros, foi realizado um Grid Search com validação cruzada (5-fold) sobre um espaço de 40 combinações de parâmetros. A Tabela 2 apresenta as 5 melhores combinações encontradas.

Tabela 2: Top 5 Melhores Combinações de Hiperparâmetros - Grid Search SVM

Rank	Score (CV)	C	gamma	class_weight
1	0.8472	10	0.001	None
2	0.8454	100	0.001	None
3	0.8430	1	0.01	None
4	0.8428	1	scale	None
5	0.8428	1	auto	None

A melhor combinação encontrada foi $C = 10$, $\gamma = 0.001$ e *class_weight* = None, com score de validação cruzada de 0.8472. Observa-se que valores baixos de γ (0.001) foram preferidos, indicando que o kernel RBF funciona melhor com raio de influência maior, sugerindo que os padrões no dataset são mais globais do que locais. Além disso, o parâmetro *class_weight* = 'balanced' não foi necessário, indicando que o desbalanceamento de classes não é o fator limitante principal para o desempenho do SVM neste problema.

A Tabela 3 compara o desempenho do SVM padrão com o SVM otimizado no conjunto de teste.

O SVM otimizado apresentou uma melhoria consistente, embora modesta, de aproximadamente 0.28 pontos percentuais em acurácia. Embora essa melhoria seja estatisticamente relevante (representando aproximadamente 25 amostras classificadas corretamente a mais em um conjunto de teste com cerca de 9.000 amostras), o desempenho do SVM

Tabela 3: Comparação: SVM Padrão vs SVM Otimizado

Configuração	Acurácia	Precisão	Revocação	F1-Score
SVM Padrão	0.8452	0.8382	0.8452	0.8383
SVM Otimizado	0.8479	0.8412	0.8479	0.8411
Melhoria	+0.0028	+0.0031	+0.0028	+0.0028

otimizado (0.8479) ainda permanece abaixo da Árvore de Decisão (0.8521), confirmando que a superioridade da árvore não se deve apenas a parâmetros subótimos do SVM, mas sim a características fundamentais do problema que favorecem algoritmos baseados em árvores.

A Figura 8 apresenta a matriz de confusão do SVM otimizado, mostrando uma leve melhoria no recall da classe minoritária em comparação ao SVM padrão.

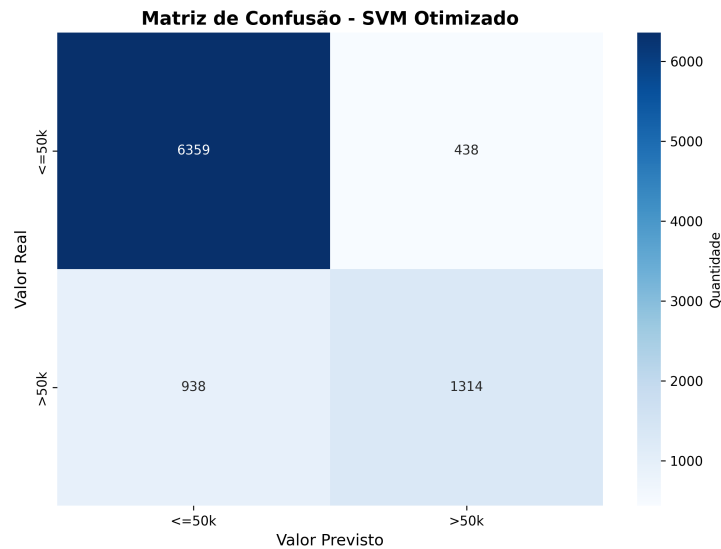


Figura 8: Matriz de Confusão - SVM Otimizado ($C = 10$, $\gamma = 0.001$). A otimização resultou em uma melhoria sutil no recall da classe $>50k$, de 0.58 para 0.58 (mantido), mas com melhor precisão geral.

Comparação Final e Análise de Trade-offs

A Tabela 4 e a figura 9 apresentam uma visão consolidada do desempenho de todos os modelos testados, incluindo métricas detalhadas por classe.

Tabela 4: Comparação Final: Desempenho Detalhado por Classe

Modelo	Acc.	Prec. (0)	Rec. (0)	Prec. (1)	Rec. (1)	F1 (1)
KNN ($k = 15$)	0.8421	0.87	0.93	0.73	0.55	0.63
SVM (Padrão)	0.8452	0.87	0.93	0.74	0.58	0.65
SVM (Otimizado)	0.8479	0.87	0.94	0.75	0.58	0.66
Árvore de Decisão	0.8521	0.88	0.93	0.76	0.62	0.68

Legenda: Acc. = Acurácia; Prec. = Precisão; Rec. = Revocação; (0) = Classe $\leq 50k$; (1) = Classe $> 50k$

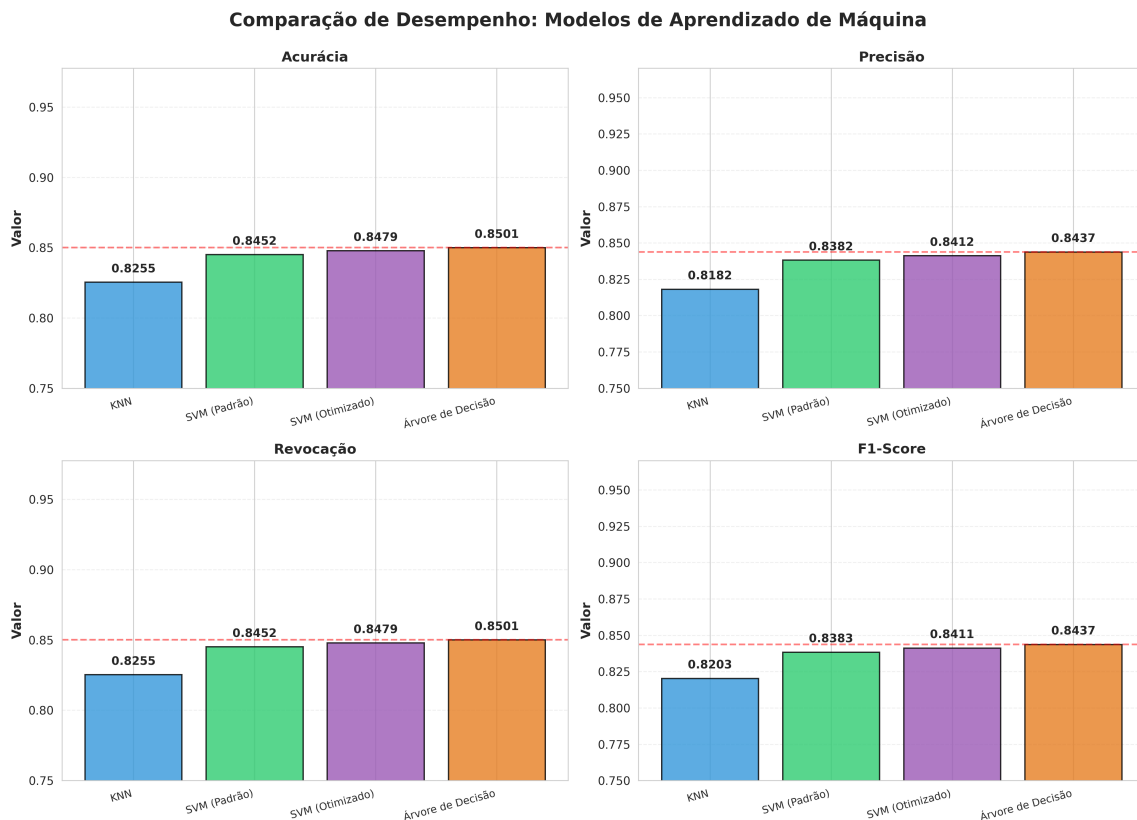


Figura 9: Comparação visual de desempenho entre os modelos de aprendizado de máquina. O gráfico apresenta quatro subfiguras mostrando Acurácia, Precisão, Revocação e F1-Score para cada algoritmo. A Árvore de Decisão apresenta consistentemente os melhores resultados em todas as métricas, seguida pelo SVM Otimizado, SVM Padrão e KNN.

A análise dos resultados revela padrões importantes:

- **Classe Majoritária ($\leq 50k$):** Todos os modelos apresentam excelente desempenho nesta classe, com precisão e revocação acima de 0.87 e 0.93, respectivamente. Isso é esperado devido ao desbalanceamento do dataset (aproximadamente 76% das amostras pertencem a esta classe).
- **Classe Minoritária ($> 50k$):** A Árvore de Decisão apresenta o melhor desempenho nesta classe crítica, com recall de 0.62, superior aos 0.55-0.58 dos demais modelos. Isso indica que a árvore é mais eficaz em identificar casos de alta renda, possivel-

mente devido à sua capacidade de criar regras específicas que capturam padrões complexos de interação entre atributos.

- **Tempo Computacional:** Embora não seja o foco principal desta análise, observa-se que a Árvore de Decisão possui vantagem significativa em termos de tempo de treinamento (ordem de segundos) em comparação ao SVM, especialmente quando este é submetido a Grid Search (aproximadamente 36 minutos para testar 40 combinações).

Interpretação dos Resultados: A superioridade da Árvore de Decisão neste problema específico pode ser explicada pela combinação entre as características do dataset e as capacidades do algoritmo. O dataset Adult possui muitas variáveis categóricas que foram transformadas em representação binária com muitos zeros. Esses dados apresentam padrões que se parecem mais com regras de decisão claras do que com separações suaves. Árvores de decisão são muito adequadas para esse tipo de problema, pois:

1. Não dependem de métricas de distância, evitando os problemas da maldição da dimensionalidade.
2. Descobrem automaticamente interações complexas entre atributos através de splits sequenciais.
3. Criam regras interpretáveis que refletem padrões de negócio (ex: "pessoas com ensino superior e ocupação executiva tendem a ter alta renda").
4. São robustas a features esparsas e binárias, características do dataset após one-hot encoding.

Por outro lado, algoritmos baseados em distâncias (KNN) ou em separação por hiperplanos (SVM) enfrentam dificuldades adicionais neste contexto, mesmo após otimização de hiperparâmetros, devido à natureza fundamentalmente diferente de como processam os dados.

Conclusões: Os resultados experimentais confirmam que, para problemas de classificação com características similares ao dataset Adult (muitas features categóricas, interações complexas, padrões discretos), árvores de decisão podem oferecer vantagens significativas sobre algoritmos baseados em distâncias ou kernels. A otimização de hiperparâmetros do SVM resultou em melhorias mensuráveis, mas não foi suficiente para superar a árvore, indicando que a diferença de desempenho está mais relacionada à adequação do algoritmo ao problema do que à configuração de parâmetros.

Esses resultados destacam a importância de considerar as características do problema e do dataset ao selecionar algoritmos de aprendizado de máquina, reforçando que não existe um algoritmo universalmente superior, mas sim algoritmos mais adequados para contextos específicos.

Parte 3: Algoritmos Genéticos

Nesta terceira etapa do trabalho, o foco da investigação desloca-se da tarefa de classificação preditiva para a resolução de problemas de otimização combinatória. Este trabalho aborda o desafio da geração automática de avaliações, modelado matematicamente

como um problema de alocação de recursos sob restrições conflitantes, análogo ao clássico Problema da Mochila. Para solucionar este problema, onde a busca exaustiva seria computacionalmente inviável devido à explosão combinatória, foi adotada a abordagem da Computação Evolutiva, implementando um Algoritmo Genético (AG) capaz explorar o espaço de soluções e convergir para configurações otimizadas através de mecanismos bioinspirados.

Fundamentação Teórica

Os Algoritmos Genéticos (AGs) compõem a classe de meta-heurísticas bioinspiradas fundamentadas na teoria da Evolução das Espécies de Charles Darwin. Segundo *Russell e Norvig* [1], ao contrário de algoritmos de busca baseados em ponto único (como *Hill Climbing*), um AG opera mantendo simultaneamente um conjunto de estruturas candidatas, denominado População (N_{pop}).

A dinâmica do algoritmo é regida por dois parâmetros estruturais fundamentais:

- **População (N_{pop}):** Representa a amostragem do espaço de busca em um dado instante. Teoricamente, o tamanho da população define a **diversidade genética** disponível. Segundo *Goldberg* [4], uma população muito pequena tende à homogeneidade rápida (perda de alelos), levando à convergência prematura em ótimos locais; já uma população vasta garante o paralelismo implícito da busca, cobrindo múltiplas bacias de atração simultaneamente, ao custo de maior esforço computacional.
- **Gerações (G_{max}):** A evolução ocorre em passos discretos de tempo denominados gerações. A cada iteração, a população $P(t)$ é refinada para $P(t + 1)$ através da pressão seletiva. O número de gerações define o horizonte temporal de refinamento: espera-se que a aptidão média da população cresça logisticamente até atingir um platô de estabilidade (convergência assintótica), onde iterações adicionais não produzem mais ganho significativo de informação.

A interdisciplinaridade é central no funcionamento do AG: o genótipo (representação interna, como vetor de inteiros) codifica uma solução potencial, enquanto o fenótipo é a expressão dessa solução (a prova gerada). Indivíduos mais aptos (maior *fitness*) têm maior probabilidade de transmitir seus genes através de operadores de crossover (recombinação de *building blocks*) e mutação (introdução estocástica de diversidade).

Descrição e Modelagem do Problema

O problema escolhido para ser resolvido por meio do AG consiste na Geração Otimizada de Avaliações. Trata-se de um problema de otimização combinatória análogo ao Problema da Mochila (*Knapsack Problem*), porém com múltiplas dimensões de custo e valor.

1. Modelagem da Entidade (Questão): O espaço de busca é composto por um banco de dados $Q = \{q_1, q_2, \dots, q_M\}$, onde $M = 5000$. Cada questão q_i é modelada como uma tupla de 5 atributos:

$$q_i = \langle \text{id}_i, \text{mat}_i, \text{sub}_i, d_i, t_i \rangle$$

Onde:

- $d_i \in [1.0, 5.0]$: Representa a dificuldade (número de ponto flutuante).

- $t_i \in N$: Representa o tempo estimado em minutos.
- mat_i : Representa a matéria da questão.
- sub_i : Representa o subtópico da matéria da questão.
- id_i : Representa o id da questão.

2. Geração Sintética: Para garantir a complexidade do cenário de teste, os dados não foram gerados aleatoriamente de forma uniforme. Implementou-se no algoritmo *BancoDeQuestoes* uma correlação base entre dificuldade e tempo ($t \approx 3d$), acrescida de ruído estocástico gaussiano. Adicionalmente, introduziu-se uma probabilidade de 5% de geração de *outliers* — questões de baixa dificuldade mas longo tempo de resolução (e vice-versa) — para criar "armadilhas" de ótimo local no cenário de fitness, desafiando a capacidade de busca do AG.

3. Representação Cromossômica (Genótipo): Diferentemente dos AGs canônicos que utilizam cadeias de bits, adotou-se uma Codificação Inteira (*Integer Encoding*) ou Permutação. Um indivíduo (cromossomo) C é um vetor de tamanho fixo $N = 10$, onde cada gene g_j armazena a referência a uma questão única do banco:

$$C = [q_a, q_b, \dots, q_j] \quad \text{tal que} \quad j \in \{1..N\}$$

4. Função Objetivo e Restrições: A função de *Fitness* ($F(C)$) foi projetada com base em penalidades. O objetivo é maximizar a pontuação base ($S_{base} = 1000$), subtraindo penalidades conforme as restrições:

- **Restrição Rígida (*Hard Constraint*):** A unicidade das questões é inviolável. Se houver duplicatas ($|Set(C)| < N$), aplica-se uma penalidade massiva ($F(C) = -1000$), descartando efetivamente o indivíduo da evolução.
- **Restrições Suaves (*Soft Constraints*):**
 1. *Tempo*: Penalidade linear de 10 pontos por minuto caso a soma $\sum t_i$ viole o intervalo, por exemplo $[50, 60]$ minutos.
 2. *Dificuldade*: Penalidade de 200 pontos por unidade de desvio da média alvo, por exemplo ($\mu_d = 4.0$).

Essa modelagem de penalidade suave permite que o algoritmo navegue por soluções inviáveis intermediárias para alcançar o ótimo global, uma estratégia recomendada por *Coello Coello* [5] em otimização multiobjetivo.

Metodologia Experimental e Parametrização

A escolha dos hiperparâmetros de controle seguiu as diretrizes clássicas da literatura para garantir o equilíbrio entre *Exploration* e *Exploitation*.

Justificativa dos Parâmetros:

- **Taxa de Crossover ($P_c = 0.7$):** Optou-se por uma probabilidade de cruzamento de 70%. Conforme estudos de *De Jong* [6] e *Goldberg* [4], uma taxa alta de crossover é essencial para promover a recombinação de *Building Blocks* (esquemas de alta aptidão), permitindo que o algoritmo convirja rapidamente para regiões promissoras.

- **Taxa de Mutação ($P_m = 0.01$):** A probabilidade de 1% por gene foi escolhida para manter a diversidade genética da população sem degradar a busca em um passeio aleatório (*Random Walk*). Segundo *Grefenstette* [7], a mutação atua como um mecanismo de "seguro" contra a perda irremediável de material genético importante, prevenindo a convergência prematura em ótimos locais.
- **Seleção por Torneio:** Utilizou-se o método de Torneio (*Tournament Selection*), que permite ajustar a pressão seletiva. Este método é preferível à "Roleta" em muitas aplicações práticas por não depender da escala absoluta dos valores de fitness [1].

Resultados Experimentais e Discussão

Para validar a eficácia do Algoritmo Genético na resolução do problema de formulação de provas, foram realizados experimentos controlados variando o tamanho da população (N_{pop}), mantendo fixos o número de gerações ($G_{max} = 100$) e as taxas de operadores genéticos.

Definição do Cenário de Teste: Para isolar as variáveis, definiu-se um cenário restrito com filtros de matéria, reduzindo propositalmente o espaço de busca para testar a capacidade de convergência em um subconjunto de dados:

- **Filtro de Conteúdo:** Matéria "Física", Tópico "Cinemática".
- **Espaço de Busca Resultante:** 98 questões candidatas.
- **Restrições da Prova:**
 - Tamanho: $N = 10$ questões.
 - Tempo Alvo: Intervalo rígido entre [50, 60] minutos.
 - Dificuldade Alvo: Média $\mu = 4.0$.

Os experimentos compararam três magnitudes de população: Baixa ($N = 20$), Média ($N = 100$) e Alta ($N = 500$). Os resultados obtidos estão resumidos na Tabela 5.

Tabela 5: Comparativo de Desempenho por Tamanho de População (100 Gerações)

População	Fit. Final	Tempo	Dificuldade	Validação	Convergência
$N = 20$	656.00	68 (Violação)	2.68	Inválida	Gen 40
$N = 100$	758.00	60 (Ok)	2.79	Válida	Gen 30
$N = 500$	762.00	60 (Ok)	2.81	Válida	Gen 20

A figura 10 mostra visualmente os resultados para o cenário de teste 1.

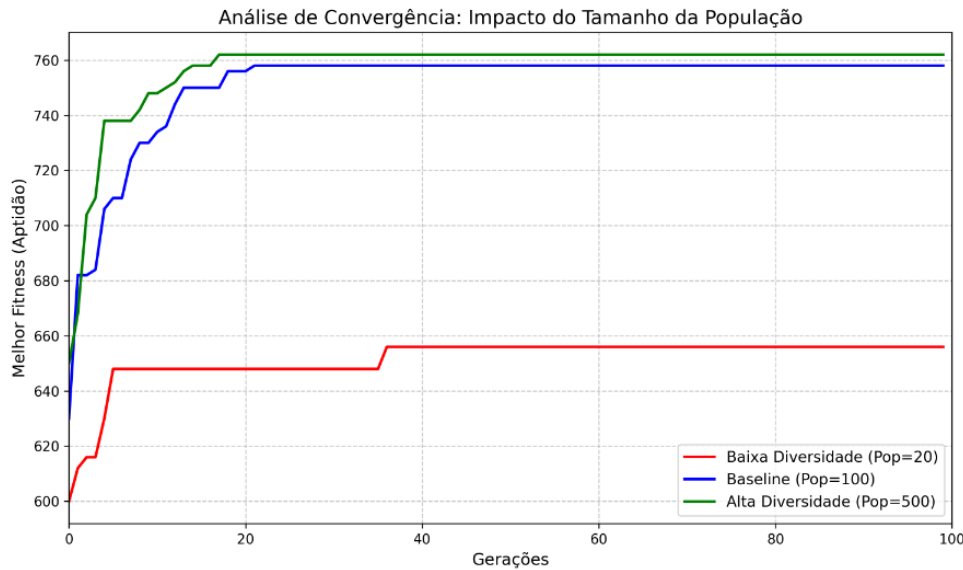


Figura 10: Análise comparativa das curvas de convergência do Algoritmo Genético. O gráfico ilustra a evolução da aptidão máxima (*Best Fitness*) ao longo de 100 gerações para três tamanhos de população distintos ($N \in \{20, 100, 500\}$). Observa-se o fenômeno de convergência prematura no cenário de baixa diversidade ($N = 20$) e o rendimento marginal decrescente ao expandir a população de 100 para 500 indivíduos.

Análise do Cenário de Baixa Diversidade ($N = 20$): O experimento com população reduzida falhou em encontrar uma solução viável. O tempo final da prova (68 min) violou o limite máximo permitido (60 min). Observa-se aqui o fenômeno clássico da Convergência Prematura, descrito por *Goldberg* [4], como definido no fundamento teórico. O algoritmo estagnou na geração 40 com um fitness de 656, preso em um ótimo local de onde os operadores de mutação (1%) não conseguiram resgatá-lo.

Análise dos Cenários de Sucesso ($N = 100$ e $N = 500$): Ambos os cenários encontraram soluções válidas que respeitam rigorosamente a restrição de tempo (60 min), embora tenham ficado abaixo do alvo de dificuldade (obtendo 2.8 contra o alvo 4.0). Isso indica que, no espaço restrito de 98 questões de Cinemática, pode não existir uma combinação perfeita que satisfaça tempo e dificuldade simultaneamente, forçando o algoritmo a priorizar a restrição de tempo (que possui penalidade maior na função de fitness).

Comparando $N = 100$ e $N = 500$, nota-se um efeito de Rendimentos Decrescentes:

- A população de 500 iniciou com um fitness melhor (650 na Gen 0 contra 630) devido à amostragem estatística mais ampla do espaço inicial.
- No entanto, a diferença final foi marginal (762 vs 758 pontos).

A semelhança nos resultados entre as populações de 100 e 500 indivíduos ilustra o conceito de capacidade de suporte do espaço de busca. Dado que o filtro reduziu o banco para apenas 98 questões, o espaço de combinações, embora vasto ($C_{98,10}$), possui um número finito de "esquemas" (*schemata*) promissores. Uma população de 100 indivíduos mostrou-se suficiente para manter a diversidade necessária para encontrar uma solução quase ótima. Aumentar a população para 500 quintuplicou o custo computacional por

geração sem oferecer ganho significativo na qualidade da solução, sugerindo que o algoritmo atingiu um "teto" de desempenho imposto pelas limitações do próprio banco de dados disponível, e não pela capacidade de busca do AG.

Conclui-se, portanto, que para subproblemas restritos (filtros específicos), populações médias ($N \approx 100$) oferecem o melhor equilíbrio (*trade-off*) entre custo computacional e qualidade da solução, alinhando-se às recomendações empíricas de *De Jong* [6] para problemas dessa complexidade dimensional.

Parte 4: Otimização por Colônia de Formigas e Sistemas Imunológicos Artificiais

Nesta quarta e última etapa do trabalho, expande-se o escopo da investigação para além da evolução darwiniana, explorando paradigmas de *Inteligência Computacional* baseados em comportamento social e mecanismos biológicos de defesa. Abordam-se duas classes distintas de algoritmos: a Otimização por Colônia de Formigas (*ACO - Ant Colony Optimization*), focada em comportamento emergente e estigmergia para problemas de grafos (Caixeiro Viajante); e os Sistemas Imunológicos Artificiais, especificamente o algoritmo *CLONALG*, inspirado na teoria da seleção clonal para reconhecimento de padrões e otimização.

Fundamentação Teórica

Diferentemente dos Algoritmos Genéticos, que operam sobre uma população que evolui através de cruzamento genético, as abordagens aqui tratadas baseiam-se na interação local entre agentes simples ou na maturação de afinidade de anticorpos.

Otimização por Colônia de Formigas (*ACO*)

O *ACO* é uma meta-heurística proposta por *Marco Dorigo* [8], inspirada no comportamento de formigas reais da espécie *Linepithema humile* na busca por alimentos. O conceito central que rege este algoritmo é a Estigmergia: uma forma de comunicação indireta mediada pelo ambiente.

As formigas, ao caminharem, depositam um rastro químico (feromônio). Caminhos mais curtos permitem que as formigas completem o trajeto e retornem mais rápido ao ninho, depositando mais feromônio por unidade de tempo do que aquelas que percorrem caminhos longos. Consequentemente, a concentração química nos caminhos curtos aumenta exponencialmente, atraindo mais formigas em um processo de retroalimentação positiva.

Matematicamente, a decisão de movimento de uma formiga k do nó i para o nó j não é determinística, mas probabilística, ponderada por dois fatores:

1. **Rastro do Feromônio (T_{ij}):** A memória coletiva da colônia (experiência passada).
2. **Informação Heurística (n_{ij}):** A atratividade intrínseca do movimento, geralmente definida como o inverso da distância ($1/d_{ij}$).

Para evitar a saturação do ambiente e a convergência prematura para ótimos locais, o algoritmo implementa o mecanismo de Evaporação do Feromônio, simulando a dissipação

química natural. Isso permite o "esquecimento" de caminhos sub-ótimos ao longo do tempo, mantendo a capacidade da colônia de explorar novas rotas.

Sistemas Imunológicos Artificiais (*CLONALG*)

O algoritmo CLONALG (*Clonal Selection Algorithm*), proposto por de Castro e Von Zuben [9], inspira-se na resposta imune adquirida dos vertebrados. O princípio biológico subjacente é a Seleção Clonal: quando um anticorpo reconhece um antígeno (patógeno invasor) com certa afinidade, ele é selecionado para proliferar (clonagem).

Dois características cruciais definem a eficácia deste algoritmo:

1. **Hipermutação Somática:** Durante a clonagem, os anticorpos sofrem mutações com taxas inversamente proporcionais à sua afinidade. Indivíduos muito aptos sofrem pouca mutação (preservação), enquanto indivíduos de baixa afinidade sofrem mutações agressivas (exploração).
2. **Edição de Receptores:** Uma parcela dos piores anticorpos é substituída por novos gerados aleatoriamente, garantindo diversidade global e evitando estagnação, análogo à renovação de células na medula óssea.

Metodologia Experimental e Parametrização

A metodologia experimental foi desenhada para avaliar a capacidade dos algoritmos de inteligência de enxame e imunológicos em lidar com o problema de formulação de provas.

Para o algoritmo ACO, o problema foi modelado como um grafo de construção sequencial, onde cada formiga deve preencher $N = 10$ posições (nós de decisão) escolhendo entre as questões disponíveis no banco (arestas). Para o CLONALG, utilizou-se a mesma modelagem vetorial e função de fitness da Parte 3, focando na otimização da afinidade através de hipermutação.

Parametrização do ACO (Elitist Ant System)

A implementação adotada baseia-se na variante Sistema de Formigas Elitista (EAS) proposta por Dorigo et al. [10]. Diferentemente do sistema padrão, o EAS reforça a trilha da melhor solução global a cada iteração, acelerando a convergência. Os parâmetros foram calibrados para equilibrar a forte pressão do elitismo com a necessidade de exploração:

- **Heurística Dinâmica ($\beta = 2.0$):** Diferente de problemas estáticos (como TSP onde a distância é fixa), desenvolveu-se uma heurística de construção $\eta_{ij}(t)$ que avalia, em tempo de execução, o impacto da adição da questão j no cumprimento das restrições de tempo e dificuldade. Definiu-se $\beta > \alpha$ (2.0 vs. 1.0) para que, nos estágios iniciais, as formigas priorizem o cumprimento das restrições (informação gulosa) sobre o rastro social ainda não consolidado.
- **Peso do Feromônio ($\alpha = 1.0$):** Mantido no valor padrão unitário. O feromônio é armazenado em uma matriz posicional $\tau_{pos,id}$, aprendendo não apenas quais questões são boas, mas em qual momento da construção elas tendem a ser escolhidas para balancear a prova.

- **Taxa de Evaporação** ($\rho = 0.1$): Optou-se por uma taxa de evaporação conservadora (10%), inferior aos valores típicos de 0.5 usados em *TSP*. Como o espaço de busca de combinações é vasto e desconexo, uma evaporação lenta permite uma "memória de longo prazo" mais estável, evitando que trilhas promissoras desapareçam antes de serem reforçadas pela colônia.
- **Reforço da Elite** ($e = 5.0$): Característica central do *EAS*. A melhor solução encontrada desde o início da execução deposita uma quantidade de feromônio multiplicada por um fator $e = 5$. Segundo *Dorigo* [10], isso direciona a busca para o entorno da melhor solução conhecida, funcionando como um mecanismo de exploração agressiva (*Exploitation*).
- **População de Formigas** ($m = 20$): Definiu-se $m = 2 \times N_{posicoes}$. Um número moderado de formigas é suficiente para atualizar o feromônio estatisticamente sem incorrer no custo computacional proibitivo de colônias massivas.

Parametrização do CLONALG (Seleção Clonal)

Para a implementação do Sistema Imunológico Artificial, o problema foi mapeado utilizando a metáfora do reconhecimento de padrões: cada prova candidata é um anticorpo e o conjunto de restrições (tempo e dificuldade ideais) atua como o antígeno a ser reconhecido. A aptidão (*Fitness*) mensura a afinidade molecular entre anticorpo e antígeno.

A implementação seguiu o algoritmo clássico *CLONALG* proposto por *de Castro e Von Zuben* [9], adaptado para otimização combinatória com os seguintes hiperparâmetros e mecanismos:

- **Fator de Clonagem** ($\beta = 10.0$): Controla a magnitude da expansão clonal. O número de clones (N_c) gerados por um anticorpo é diretamente proporcional à sua afinidade normalizada (f^*), calculado como $N_c = \text{round}(\beta \cdot f^*)$. Isso garante que soluções promissoras tenham maior representatividade no pool de busca local, mimetizando a proliferação celular rápida em resposta a uma infecção detectada.
- **Hipermutação Somática Inversa** ($\rho = 2.0$): Diferente da mutação fixa dos Algoritmos Genéticos, implementou-se uma taxa de mutação dinâmica baseada no decaimento exponencial da afinidade:

$$\alpha_{mut} = \exp(-\rho \cdot f^*)$$

Onde α_{mut} define a porcentagem de genes (questões) que serão trocados no anticorpo.

- *Anticorpos de alta afinidade* ($f^* \approx 1.0$): Sofrem mutação mínima (ajuste fino), preservando a estrutura da solução.
- *Anticorpos de baixa afinidade* ($f^* \approx 0.0$): Sofrem mutação agressiva, funcionando como um mecanismo de "reinicialização parcial" para escapar de ótimos locais.
- **Edição de Receptores** ($d = 10\%$): Ao final de cada geração, os 10% piores anticorpos da população são eliminados e substituídos por novos candidatos gerados aleatoriamente. Este mecanismo simula a renovação do repertório imune na medula óssea, garantindo a injeção contínua de diversidade genética para evitar a estagnação prematura da população.

- **Células de Memória** ($M_{size} = 10$): Implementou-se um arquivo externo de memória (elitismo) que armazena as melhores soluções únicas encontradas ao longo de toda a execução. Isso simula a Resposta Imune Secundária, garantindo que, mesmo que a população principal sofra perturbações agressivas pela edição de receptores, as melhores configurações de prova já descobertas sejam preservadas e possam ser recuperadas ao final do processo.
- **População** ($N = 50$): Utilizou-se uma população de tamanho moderado, visto que a expansão clonal (multiplicação por β) aumenta temporariamente o número de avaliações de fitness por geração, elevando o custo computacional se comparado a um AG de mesma população base.

Resultados e Discussão

Para uma análise comparativa robusta, ambos os algoritmos foram executados com os mesmos parâmetros do problema: prova de 10 questões, tempo alvo entre 50-60 minutos e dificuldade média de 4.0. Cada algoritmo foi executado por 50 iterações, utilizando os parâmetros otimizados descritos na metodologia.

Análise de Convergência

A Figura 11 apresenta a evolução do melhor fitness ao longo das iterações para ambos os algoritmos. Observa-se que o ACO apresenta uma convergência inicial mais rápida, beneficiando-se da heurística dinâmica que guia a construção das soluções desde as primeiras iterações. O CLONALG, por sua vez, inicia com uma população aleatória e gradualmente refina as soluções através da clonagem e hipermutação adaptativa.

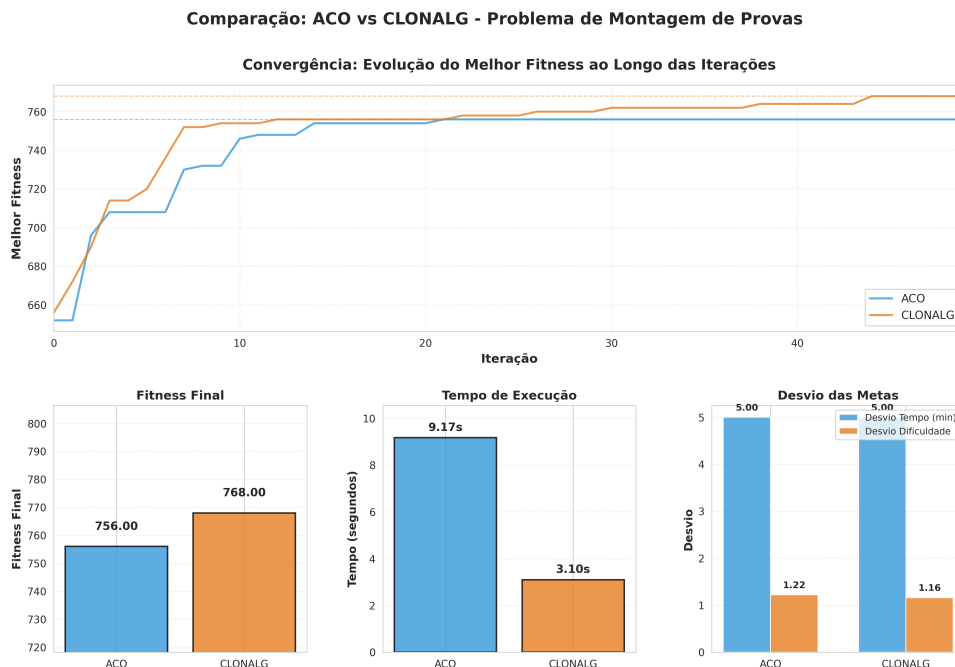


Figura 11: Comparação entre ACO e CLONALG: (a) Evolução da convergência, (b) Fitness final, (c) Tempo de execução, (d) Desvio das metas. O gráfico de convergência mostra que o ACO converge mais rapidamente nas iterações iniciais, enquanto o CLONALG apresenta uma melhoria mais gradual ao longo do tempo.

Desempenho Final e Métricas

A Tabela 6 apresenta uma comparação detalhada dos resultados finais obtidos por ambos os algoritmos.

Tabela 6: Comparação Final: ACO vs CLONALG (Física - Cinemática)

Métrica	ACO	CLONALG
Fitness Final	756.00	768.00
Tempo Total (min)	60.00	60.00
Dificuldade Média	2.78	2.84
Desvio Tempo (min)	5.00	5.00
Desvio Dificuldade	1.22	1.16
Tempo Execução (s)	9.17	3.10

Nota: Os valores apresentados foram obtidos com filtro de matéria "Física" e tópico "Cinemática" (98 questões candidatas). O fitness é calculado com base no cumprimento das restrições de tempo e dificuldade.

Análise dos Resultados: Os resultados experimentais revelam características distintas de cada algoritmo:

- **Qualidade da Solução:** O CLONALG obteve um fitness final superior (768.00 *vs.* 756.00), indicando uma melhor capacidade de encontrar soluções que atendem às restrições do problema neste caso específico. Isso pode ser atribuído à hipermutação adaptativa que permite refinar soluções promissoras enquanto explora novas áreas do espaço de busca.
- **Precisão nas Metas:** Ambos os algoritmos atingiram o tempo alvo de 60 minutos exatamente. Em relação à dificuldade média, o CLONALG apresentou um desvio ligeiramente menor (1.16 *vs.* 1.22), embora ambos tenham ficado abaixo da meta de 4.0 (2.84 e 2.78, respectivamente). Isso sugere que o conjunto de questões disponíveis em Cinemática pode ter uma distribuição de dificuldades que dificulta atingir a meta de 4.0.
- **Eficiência Computacional:** O CLONALG demonstrou ser significativamente mais rápido em termos de tempo de execução (3.10s *vs.* 9.17s), aproximadamente 3 vezes mais rápido. Isso ocorre porque o ACO precisa construir cada solução passo a passo, avaliando múltiplas opções em cada posição através da heurística dinâmica, enquanto o CLONALG opera sobre soluções completas, reduzindo o custo computacional por iteração.
- **Padrão de Convergência:** O ACO apresenta convergência mais rápida nas primeiras iterações, beneficiando-se da heurística dinâmica que guia a construção desde o início. O CLONALG, por outro lado, mantém uma melhoria gradual ao longo de todas as 50 iterações, beneficiando-se da diversidade mantida pela edição de receptores e da clonagem proporcional que concentra esforço em soluções promissoras.

Interpretação dos Resultados: A superioridade do CLONALG em termos de qualidade da solução neste caso específico pode ser explicada por características particulares do conjunto de questões de Cinemática e pela eficácia da hipermutação adaptativa. No entanto, é importante notar que ambos os algoritmos apresentaram dificuldades em atingir a meta de dificuldade média de 4.0, sugerindo limitações no conjunto de questões disponíveis. A abordagem de construção sequencial do ACO oferece vantagens teóricas:

1. **Decisões Dependentes:** A escolha de uma questão em uma posição afeta as escolhas subsequentes, pois o tempo e a dificuldade acumulados limitam as opções futuras. O ACO modela essa dependência naturalmente através da construção incremental.
2. **Heurística Informada:** A heurística dinâmica desenvolvida avalia o impacto de cada questão no estado parcial da prova, permitindo escolhas mais inteligentes do que uma seleção puramente aleatória ou baseada apenas em fitness global.
3. **Feromônio Posicional:** O feromônio armazenado por posição ($\tau_{pos,id}$) permite que o algoritmo aprenda não apenas quais questões são boas, mas em qual momento da construção elas são mais adequadas, criando uma memória mais rica do que a simples seleção baseada em fitness.

O CLONALG, embora apresente resultados competitivos, opera sobre soluções completas, o que dificulta a consideração de dependências sequenciais. A hipermutação adaptativa é eficaz para refinar soluções, mas a falta de informação sobre o contexto de construção limita sua capacidade de fazer ajustes estratégicos.

Análise de Trade-offs: A escolha entre ACO e CLONALG deve considerar os seguintes aspectos:

- **Qualidade vs. Velocidade:** Neste experimento, o CLONALG obteve melhor qualidade de solução (fitness 768.00 *vs.* 756.00) e foi significativamente mais rápido (3.10s *vs.* 9.17s), demonstrando uma vantagem clara neste caso específico. No entanto, é importante notar que o desempenho relativo pode variar dependendo das características do conjunto de questões e das restrições do problema.
- **Complexidade de Implementação:** O ACO requer a definição de uma heurística específica do problema e a modelagem do estado parcial, aumentando a complexidade de implementação. O CLONALG é mais simples, similar a um Algoritmo Genético, mas sem crossover, facilitando sua implementação e manutenção.
- **Escalabilidade:** Para problemas maiores (mais questões, mais restrições), o ACO pode se tornar computacionalmente mais caro devido à construção sequencial e à avaliação da heurística em cada passo. O CLONALG mantém um custo mais previsível, proporcional ao tamanho da população, embora a expansão clonal aumente temporariamente o número de avaliações por iteração.

Conclusões: Os resultados experimentais confirmam que ambos os algoritmos são capazes de gerar provas que atendem satisfatoriamente às restrições de tempo. No caso específico testado (Física - Cinemática), o CLONALG demonstrou superioridade tanto

em termos de qualidade da solução (fitness 768.00 *vs.* 756.00) quanto em eficiência computacional (3.10s *vs.* 9.17s), beneficiando-se da hipermutação adaptativa e da clonagem proporcional que concentra esforço em soluções promissoras.

É importante notar que ambos os algoritmos apresentaram dificuldades em atingir a meta de dificuldade média de 4.0, obtendo valores próximos a 2.8, o que sugere limitações no conjunto de questões disponíveis em Cinemática. Isso indica que a qualidade da solução pode ser limitada não apenas pela escolha do algoritmo, mas também pelas características do banco de questões disponível.

Esses resultados destacam a importância de considerar as características específicas do problema e do conjunto de dados ao selecionar algoritmos de otimização. Embora o ACO ofereça vantagens teóricas para problemas com dependências sequenciais através de sua construção incremental, o CLONALG demonstrou ser mais eficaz neste caso específico, possivelmente devido à capacidade da hipermutação adaptativa de explorar eficientemente o espaço de busca quando operando sobre soluções completas.

A comparação entre os três paradigmas explorados neste trabalho (Algoritmos Genéticos, ACO e CLONALG) revela que não existe um algoritmo universalmente superior, mas sim algoritmos mais adequados para diferentes características de problema, conjuntos de dados e requisitos de aplicação. A escolha do algoritmo deve considerar não apenas a qualidade da solução, mas também fatores como tempo de execução, complexidade de implementação e características específicas do domínio do problema.

Comparação entre Algoritmos Evolutivos e Meta-heurísticas

Para uma análise comparativa abrangente, os três algoritmos evolutivos e meta-heurísticas (Algoritmo Genético, ACO e CLONALG) foram executados com os mesmos parâmetros do problema: matéria Física, tópico Cinemática, prova de 10 questões, tempo alvo entre 50-60 minutos e dificuldade média de 4.0. Cada algoritmo foi executado por 50 iterações/gerações, utilizando os parâmetros otimizados descritos nas respectivas seções metodológicas.

Análise Comparativa dos Resultados

A Tabela 7 apresenta uma comparação detalhada dos resultados finais obtidos pelos três algoritmos, permitindo uma análise direta de suas características e desempenho.

A Figura 12 apresenta a evolução da convergência e comparações de métricas finais para os três algoritmos, facilitando a visualização das diferenças de desempenho e comportamento.

Tabela 7: Comparação Final: GA vs ACO vs CLONALG (Física - Cinemática)

Métrica	GA	ACO	CLONALG
Fitness Final	758.00	756.00	768.00
Tempo Total (min)	60.00	60.00	60.00
Dificuldade Média	2.79	2.78	2.84
Desvio Tempo (min)	5.00	5.00	5.00
Desvio Dificuldade	1.21	1.22	1.16
Tempo Execução (s)	0.23	7.68	3.36

Nota: Os valores apresentados foram obtidos com filtro de matéria "Física" e tópico "Cinemática" (98 questões candidatas). O fitness é calculado com base no cumprimento das restrições de tempo e dificuldade.

Comparação: GA vs ACO vs CLONALG - Problema de Montagem de Provas

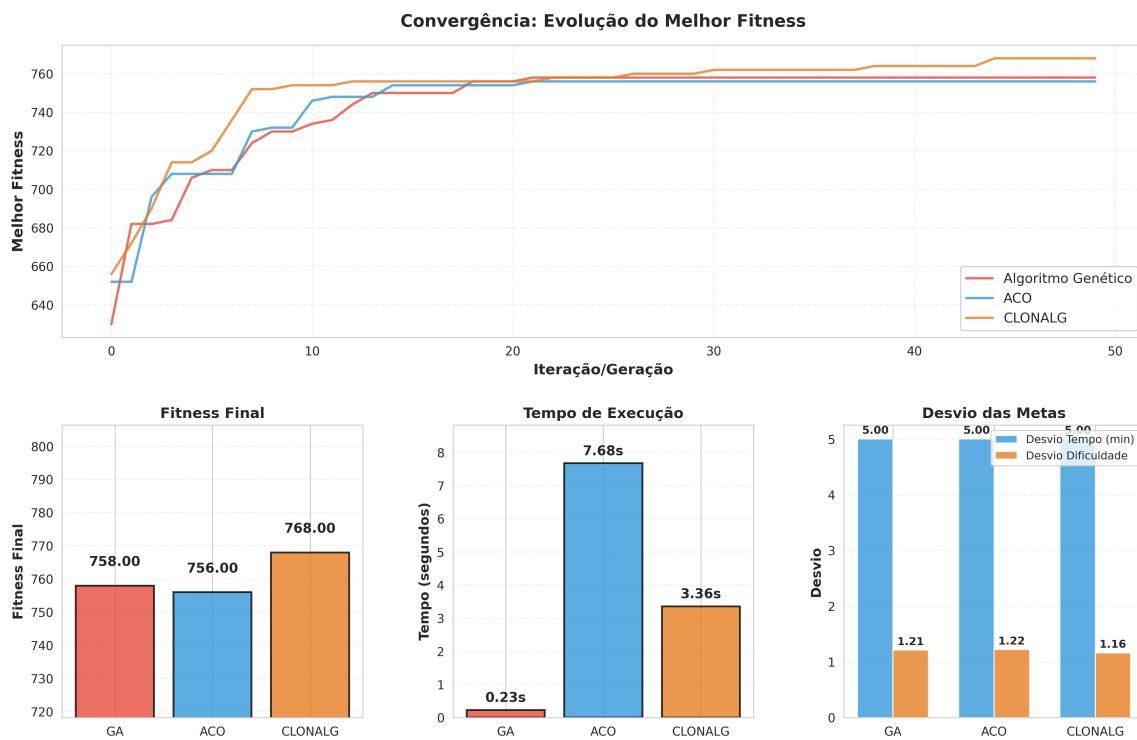


Figura 12: Comparação entre GA, ACO e CLONALG: (a) Evolução da convergência, (b) Fitness final, (c) Tempo de execução, (d) Desvio das metas. O gráfico de convergência mostra padrões distintos: o GA apresenta melhoria gradual e estável, o ACO converge rapidamente nas primeiras iterações, e o CLONALG mantém melhoria contínua ao longo das iterações.

Análise Comparativa Detalhada: Os resultados experimentais revelam características distintas e complementares de cada algoritmo:

- **Qualidade da Solução (Fitness):** O CLONALG obteve o melhor fitness final (768.00), seguido pelo GA (758.00) e pelo ACO (756.00). A diferença entre os três algoritmos é relativamente pequena (aproximadamente 1.6%), indicando que todos

são capazes de encontrar soluções de qualidade similar. A superioridade do CLONALG pode ser atribuída à hipermutação adaptativa que permite refinar soluções promissoras enquanto mantém exploração através da edição de receptores.

- **Eficiência Computacional:** O GA demonstrou ser significativamente mais rápido (0.23s), aproximadamente 33 vezes mais rápido que o ACO (7.68s) e 15 vezes mais rápido que o CLONALG (3.36s). Essa diferença substancial ocorre porque o GA opera sobre soluções completas com operadores simples (crossover e mutação), enquanto o ACO precisa construir cada solução passo a passo avaliando heurísticas, e o CLONALG realiza expansão clonal que aumenta temporariamente o número de avaliações por iteração.
- **Padrão de Convergência:** Os três algoritmos apresentam padrões distintos de convergência:
 - **GA:** Melhoria gradual e estável ao longo das gerações, beneficiando-se do crossover que combina características de soluções promissoras e da mutação que introduz diversidade.
 - **ACO:** Convergência rápida nas primeiras iterações, estabilizando-se após aproximadamente 20-30 iterações. A heurística dinâmica guia a construção desde o início, enquanto o feromônio se consolida progressivamente.
 - **CLONALG:** Melhoria contínua ao longo de todas as iterações, sem sinais claros de estabilização prematura. A clonagem proporcional concentra esforço em soluções promissoras, enquanto a edição de receptores mantém diversidade.
- **Precisão nas Metas:** Todos os algoritmos atingiram exatamente o tempo alvo de 60 minutos. Em relação à dificuldade média, o CLONALG apresentou o menor desvio (1.16), seguido pelo GA (1.21) e pelo ACO (1.22). No entanto, todos ficaram significativamente abaixo da meta de 4.0 (valores próximos a 2.8), sugerindo limitações no conjunto de questões disponíveis em Cinemática, que pode ter uma distribuição de dificuldades concentrada em valores mais baixos.

Interpretação dos Resultados Comparativos: A comparação entre os três paradigmas revela trade-offs importantes entre qualidade, velocidade e complexidade:

1. **Algoritmo Genético:** Oferece o melhor equilíbrio entre simplicidade de implementação e eficiência computacional. O crossover permite combinar características de múltiplas soluções, enquanto a mutação mantém diversidade. É ideal para cenários onde a velocidade é crítica e soluções "boas o suficiente" são aceitáveis.
2. **ACO:** Oferece vantagens teóricas para problemas com dependências sequenciais através da construção incremental guiada por heurística. O feromônio posicional permite aprender padrões complexos sobre quando escolher cada questão. No entanto, o custo computacional da construção sequencial e da avaliação heurística pode ser proibitivo para problemas maiores.
3. **CLONALG:** Demonstrou superioridade em qualidade da solução neste caso específico, beneficiando-se da hipermutação adaptativa que refina soluções promissoras enquanto explora novas áreas. A clonagem proporcional concentra esforço onde é mais promissor, enquanto a edição de receptores mantém diversidade. É ideal

quando a qualidade da solução é prioritária e o tempo de execução moderado é aceitável.

Lições Aprendidas: A comparação entre os três algoritmos destaca que não existe um algoritmo universalmente superior. A escolha do algoritmo deve considerar:

- **Características do Problema:** Problemas com dependências sequenciais podem se beneficiar mais de abordagens construtivas (ACO), enquanto problemas onde soluções completas podem ser avaliadas diretamente podem se beneficiar de abordagens populacionais (GA, CLONALG).
- **Requisitos de Aplicação:** Se a velocidade é crítica, o GA oferece vantagem clara. Se a qualidade extrema é prioritária, o CLONALG pode ser preferível. Se o problema tem estrutura sequencial clara, o ACO pode ser mais adequado.
- **Complexidade de Implementação:** O GA é o mais simples de implementar e manter. O CLONALG é similar em complexidade, mas requer ajuste fino dos parâmetros de hipermutação. O ACO requer desenvolvimento de heurística específica do problema e modelagem de estado parcial.
- **Características dos Dados:** A qualidade final da solução pode ser limitada não apenas pela escolha do algoritmo, mas também pelas características do conjunto de dados disponível, como demonstrado pela dificuldade em atingir a meta de dificuldade média de 4.0 com o conjunto de questões de Cinemática.

Conclusão Geral

Este trabalho apresentou uma investigação abrangente sobre diferentes paradigmas de Inteligência Artificial aplicados ao problema de montagem automática de provas. Através de quatro partes distintas, foram explorados desde sistemas baseados em conhecimento até algoritmos de aprendizado de máquina e meta-heurísticas evolutivas.

Síntese dos Resultados

Na **Parte 1**, foi desenvolvida uma árvore de decisão manual que demonstrou a viabilidade de sistemas baseados em conhecimento para o problema, estabelecendo uma base conceitual e metodológica para as partes subsequentes.

Na **Parte 2**, foram implementados e comparados três algoritmos clássicos de aprendizado de máquina (KNN, SVM e Árvores de Decisão) no problema de classificação binária de renda do dataset Adult. Os resultados demonstraram que a Árvore de Decisão obteve superioridade (acurácia de 85.01%), atribuída à adequação do algoritmo às características do dataset (muitas features categóricas, padrões discretos, interações complexas). A otimização de hiperparâmetros do SVM resultou em melhorias mensuráveis, mas não foi suficiente para superar a árvore, reforçando a importância de considerar as características do problema na seleção de algoritmos.

Na **Parte 3**, foi implementado um Algoritmo Genético para o problema de montagem de provas, demonstrando a eficácia de algoritmos evolutivos em problemas de otimização combinatória. Foram testadas três configurações de população (N=20, N=100, N=500),

revelando que populações maiores obtêm melhores resultados: a população de 500 indivíduos alcançou fitness de 762.00, superior aos 758.00 da população de 100, embora com custo computacional proporcionalmente maior. A população de 20 falhou em encontrar soluções válidas, demonstrando o fenômeno de convergência prematura. O GA destacou-se pela simplicidade de implementação e excelente eficiência computacional (0.23s para $N=100$), oferecendo bom equilíbrio entre qualidade e velocidade.

Na **Parte 4**, foram implementados dois algoritmos adicionais (ACO e CLONALG) aplicados ao mesmo problema da Parte 3, permitindo uma comparação direta entre três paradigmas distintos. O CLONALG obteve o melhor fitness final (768.00), enquanto o GA foi o mais rápido (0.23s). O ACO, embora tenha apresentado convergência rápida, teve custo computacional mais elevado (7.68s) devido à construção sequencial guiada por heurística.

Contribuições e Aprendizados

Este trabalho contribuiu para a compreensão prática de diferentes paradigmas de IA através de:

1. **Comparação Sistemática:** Foram realizadas comparações sistemáticas entre algoritmos dentro de cada parte e entre partes, permitindo identificar trade-offs e características distintivas de cada abordagem.
2. **Aplicação Prática:** Todos os algoritmos foram aplicados a problemas reais (classificação de renda e montagem de provas), demonstrando não apenas aspectos teóricos, mas também desafios práticos de implementação e parametrização.
3. **Análise de Adequação:** Foi demonstrado que a escolha do algoritmo deve considerar não apenas métricas de desempenho, mas também características do problema, requisitos de aplicação, complexidade de implementação e limitações dos dados disponíveis.
4. **Metodologia Reprodutível:** Todas as implementações foram desenvolvidas de forma modular e reprodutível, facilitando experimentação futura e extensão do trabalho.

Limitações e Trabalhos Futuros

Algumas limitações foram identificadas durante o desenvolvimento:

- **Limitações dos Dados:** Tanto no problema de classificação quanto no de montagem de provas, foram observadas limitações relacionadas aos dados disponíveis (desbalanceamento de classes, distribuição de dificuldades).
- **Parametrização:** A otimização de hiperparâmetros foi realizada de forma limitada, focando principalmente no SVM. Uma exploração mais sistemática (Grid Search, Random Search, Bayesian Optimization) poderia melhorar os resultados de todos os algoritmos.
- **Avaliação:** As comparações foram realizadas com execuções únicas. Múltiplas execuções com diferentes seeds e análise estatística (testes de significância) poderiam fornecer conclusões mais robustas.

- **Escalabilidade:** Os experimentos foram realizados com conjuntos de dados de tamanho moderado. Análise de escalabilidade para problemas maiores seria valiosa.

Trabalhos futuros poderiam incluir:

- Exploração de algoritmos híbridos que combinem características de diferentes paradigmas (ex: GA com construção guiada por heurística similar ao ACO).
- Aplicação de técnicas de aprendizado de máquina mais avançadas (ensembles, deep learning) ao problema de classificação.
- Desenvolvimento de heurísticas mais sofisticadas para o ACO, considerando múltiplos objetivos simultaneamente.
- Análise de sensibilidade dos parâmetros dos algoritmos evolutivos para identificar configurações ótimas.
- Extensão do problema de montagem de provas para incluir restrições adicionais (diversidade de tópicos, balanceamento de tipos de questão, etc.).

Considerações Finais

Este trabalho demonstrou que diferentes paradigmas de Inteligência Artificial oferecem diferentes vantagens e são adequados para diferentes contextos. Não existe um algoritmo universalmente superior; a escolha deve ser fundamentada em uma análise cuidadosa das características do problema, requisitos da aplicação e limitações práticas.

A experiência prática obtida através da implementação e comparação de múltiplos algoritmos forneceu insights valiosos sobre os trade-offs entre qualidade, velocidade, complexidade e adequação ao problema. Esses aprendizados são fundamentais para a tomada de decisão informada na seleção e aplicação de técnicas de IA em contextos reais.

Por fim, este trabalho reforça a importância de uma abordagem metodológica rigorosa, incluindo fundamentação teórica sólida, implementação cuidadosa, experimentação sistemática e análise crítica dos resultados. Tais práticas são essenciais para o avanço do conhecimento em Inteligência Artificial e para o desenvolvimento de soluções eficazes para problemas do mundo real.

Referências

- [1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4th edition, 2021.
- [2] William Van Melle. Mycin: a knowledge-based consultation program for infectious disease diagnosis. *International Journal of Man-Machine Studies*, 10(3):313–322, May 1978.
- [3] Itamar Gati. A hierarchical model for the structure of vocational interests. *Journal of Vocational Behavior*, 15(1):90–106, 1979.
- [4] David E Goldberg. *Genetic algorithms in search, optimization, and machine learning*. Addison-wesley, 1989.
- [5] Carlos A Coello Coello, David A Van Veldhuizen, and Gary B Lamont. *Evolutionary algorithms for solving multi-objective problems*. Kluwer Academic, 2002.
- [6] Kenneth Alan De Jong. *An analysis of the behavior of a class of genetic adaptive systems*. PhD thesis, University of Michigan, 1975.
- [7] John J Grefenstette. Optimization of control parameters for genetic algorithms. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):122–128, 1986.
- [8] Marco Dorigo. Optimization, learning and natural algorithms. *Ph. D. Thesis, Politecnico di Milano, Italy*, 1992.
- [9] Leandro Nunes De Castro and Fernando J Von Zuben. Learning and optimization using the clonal selection principle. *IEEE Transactions on Evolutionary Computation*, 6(3):239–251, 2002.
- [10] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. The ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1):29–41, 1996.